



January, 2025

APM_5AI29_TP

Language Models and Structured Data

Mid-term Project Report

Acronym of the Team: VNAI

Name: Vietnamese AI Team

Reasoning and translating to First-Order Logic using Large Language Models on FOLIO

Abstract

First-Order Logic (FOL) is widely used for symbolic reasoning and logical entailment. Recent advances in large language models (LLMs) have enabled new approaches to tackling various FOL benchmarks and datasets. However, existing models still lack robust mechanisms for FOL reasoning, and NL-FOL translations often suffer from syntactic errors and failures to preserve semantic meaning. The objective of this work is to apply different LLM-based techniques to improve performance under current evaluation metrics. To this end, we introduce two methods addressing two major challenges in FOL reasoning, namely SeRAG and ST-RAG. Experimental results show that applying different prompting strategies and incorporating step-by-step translation examples can substantially improve overall performance.

1 Introduction

First-order logic (FOL) is a foundational formalism for representing structured knowledge and performing logical inference in artificial intelligence. By using objects, predicates, quantifiers, and logical connectives, FOL enables precise and unambiguous expression of facts and relationships. This property makes it especially valuable for tasks that require correctness and interpretability, such as knowledge base construction, automated theorem proving, and formal verification. As a result, FOL has long been a core research topic in symbolic AI and remains highly relevant in modern AI systems.

With the rise of large language models (LLMs), there is growing interest in using them for logical reasoning and autoformalization, translating natural language into formal logic [1, 2, 3, 4]. These tasks remain challenging: reasoning requires strict correctness, and autoformalization must handle ambiguity and complex structures. Prior methods using prompts, in-context examples and self-refinement [4, 3] show some improvement but often rely on fixed examples and lack dynamic guidance, limiting generalization and sometimes producing fluent but logically incorrect outputs.

To address these limitations, this work introduces SeRAG and ST-RAG, based on the publication [1], where ST-RAG is a thought-guided Retrieval-Augmented Generation framework that enhances autoformalization and self-refinement by constructing knowledge bases of reasoning examples, meanwhile SeRAG implements few-shot prompting from simple example sets for logic reasoning. We evaluate our approach on the FOLIO dataset [5], a benchmark for first-order logical reasoning from natural language.

2 Related Works

Translating natural language (NL) into formal languages is challenging due to ambiguity and implicit information in NL. Early studies [6, 7] combined manual rules with deep learning or applied supervised and reinforcement learning to improve translation quality. Other works [8] explored intermediate representations to bridge the gap between NL and formal logic.

Existing chain-of-thought-based logical reasoning pipelines [2] add more implicit logic reasoning derived from the input premises. But in many cases, the refinement step does not provide additional knowledge or guidance, resulting in limited improvement despite doubling the computational cost due to multiple LLM calls. Furthermore, these approaches rely on static prompts or fixed examples, which limits their effectiveness on complex inputs.

Recently, autoformalization-based neuro-symbolic methods for logical reasoning have attracted significant attention. Prior approaches typically formalize NL inputs into logical forms and then apply symbolic solvers [4]. Other methods self-refinement mechanisms, majority-voting aggregation, or the use of SAT solvers and Lean theorem provers [3]. There is the study which fine-tuned models within formal environments [9], while others rely on chain-of-thought prompting over both NL and formal representations [10].

In parallel, Retrieval-Augmented Generation (RAG) [11] has been proposed to enhance generation quality by incorporating external knowledge through retrieval. Subsequent work [12] shows that RAG can reduce hallucinations and improve output reliability, especially when retrieval is efficiently integrated during generation. The paper [13] has also showed dynamic and diversity-aware selection strategies are critical for in-context learning and consistently outperformed fixed example sets.

3 Problem Formulation

We consider two related tasks defined in the FOLIO dataset for evaluating first-order logic reasoning from natural language: 1) logical reasoning, 2) natural language to first-order logic translation (NL-FOL).

3.1 Logical Reasoning

Given a set of natural language (NL) *premises* $\mathbf{P} = \{p_1, \dots, p_n\}$ and a natural language *conclusion* $\mathbf{h}$, the objective is to determine the logical truth value of the conclusion under the world defined by the premises. Specifically, the model must classify the conclusion into one of the labels in $\mathbf{Y} = \{\text{True}, \text{False}, \text{Unknown}\}$. Formally, the task can be defined as learning a function

$$f_{\text{reason}} : (\mathbf{P}, \mathbf{h}) \rightarrow y \in \mathcal{Y} \quad (1)$$

where the predicted label y should match the deductive inference based on logical consistency with the given premises.

3.2 NL-FOL Translation

In addition to reasoning over natural language, FOLIO defines a translation task between natural language and formal first-order logic representations. Each NL premise p_i and conclusion $\mathbf{h}$ has a corresponding FOL formula pf_i and $\mathbf{hf}$ that represent their precise logical meaning. The goal is to translate an NL story $S = (\mathbf{P}, \mathbf{hf})$ into a set of FOL formulas $F = (\{pf_1, \dots, pf_n\}, \mathbf{hf})$ such that the semantics of the NL input are preserved. The translation model learns a function

$$f_{\text{NL-FOL}} : S \rightarrow F \quad (2)$$

where each generated formula in F should be logically and semantically equivalent to its NL counterpart. Moreover, the truth value of the conclusion under the translated FOL representation should be identical to that under the original NL story.

4 Method/Overall Architecture

To address the two target problems, two models are introduced.

4.1 Semantic RAG (SeRAG)

We first introduce SeRAG, which is designed to tackle the problem of natural language reasoning with FOL. A naive approach that directly feeds the premises and conclusion in natural language into a model to predict *True*, *False*, or *Uncertain* is overly simplistic and fails to exploit available reasoning patterns.

To overcome this limitation, we incorporate a RAG module to enable few-shot prompting. In this setting, the knowledge base consists of examples extracted from the training set. Given an input query, the agent retrieves the k nearest neighbors in the embedding space, which serve as representative reasoning examples. The overall architecture of SeRAG is illustrated in Figure 1.

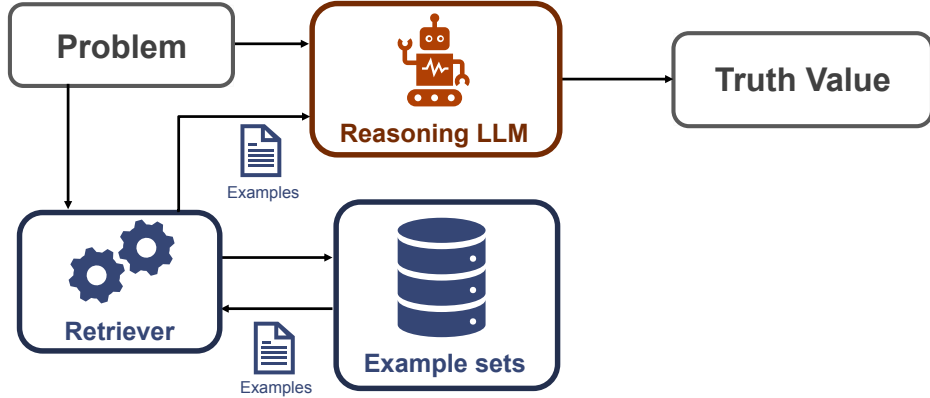


Figure 1: SeRAG.

To further improve retrieval efficiency, we employ FAISS [14] as a supporting indexing mechanism, significantly reducing the search time in the vector space. The retrieval process using FAISS is illustrated in Figure 2.

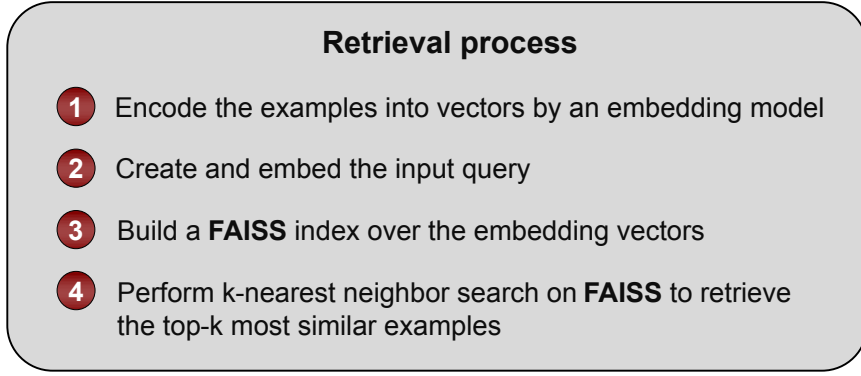


Figure 2: Retrieval process applying FAISS.

4.2 Step-by-step Translation based on RAG (ST-RAG)

The second task, translating natural language into FOL (NL-FOL Translation), is considerably more challenging. Our approach for this task is inherited from SeRAG, where a RAG module is employed to retrieve relevant examples and support few-shot prompting.

However, we observe that relying solely on example pairs consisting of natural language premises and their corresponding FOL representations is insufficient for accurate translation. To address this issue, we enhance the example set by manually incorporating step-by-step translation guidelines into each retrieved example, as illustrated in Figure 3.

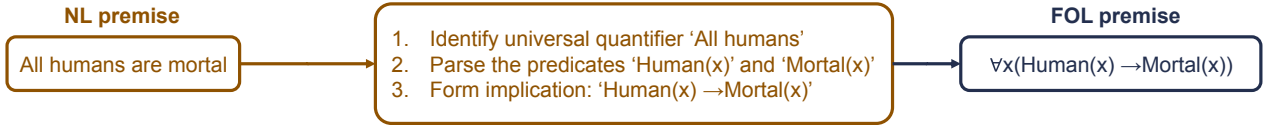


Figure 3: Retrieval process applying FAISS.

Once the FOL translation is generated, it is passed to an exact solver. The solver uses regular expressions [15] to parse the FOL expressions, translate them into executable Python code, and then evaluate them. Two outcomes are possible: (i) successful execution, where the solver evaluates the premises and outputs the truth value of the conclusion; or (ii) failure, where parsing or execution errors occur.

In the latter case, the erroneous FOL expression and the corresponding error message are forwarded to a fixer module, which attempts to correct the translation. This fixing process iterates until no errors remain or a predefined maximum number of attempts is reached. The overall workflow of ST-RAG is illustrated in Figure 4.

5 Experimentation

5.1 Evaluation Metrics

We evaluate the two tasks in FOLIO [5] using the following metrics:

5.1.1 Logical Reasoning

Accuracy For logical reasoning, we use classification accuracy (Acc) to measure how often the predicted label matches the ground-truth label:

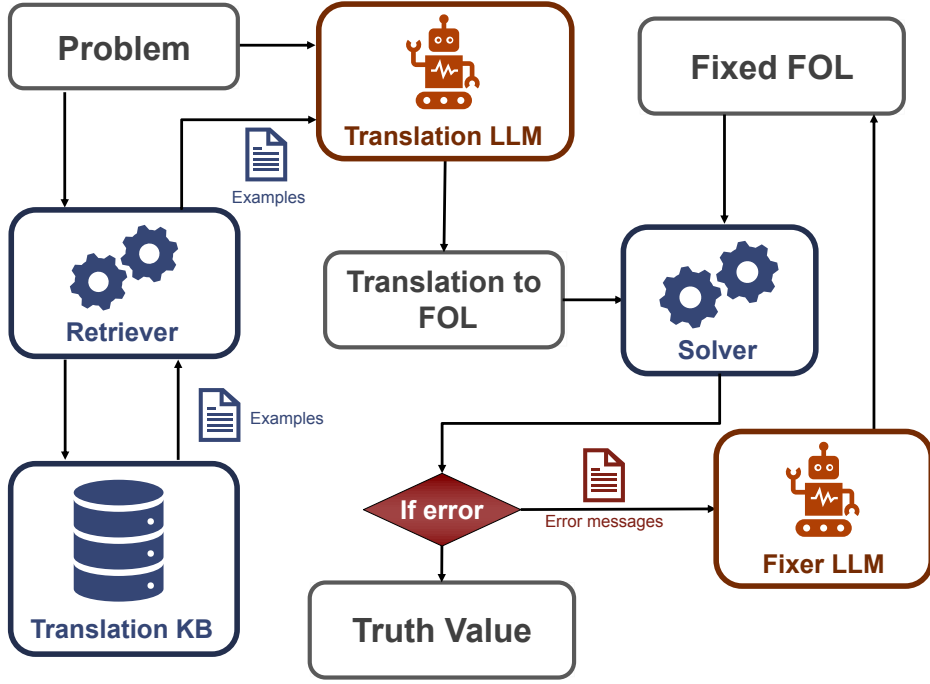


Figure 4: Retrieval process applying FAISS.

$$\text{Acc} = \frac{\# \text{ correct predictions}}{\text{total } \# \text{ of examples}} \quad (3)$$

5.1.2 NL-FOL Translation

For NL-to-FOL translation, we consider two metrics:

Syntax Validity (SynV) Measures the proportion of generated FOL formulas that are syntactically valid (i.e., contain no parsing or syntax errors):

$$\text{SynV} = \frac{\# \text{ syntactically valid generated FOL formulas}}{\text{total } \# \text{ of examples}} \quad (4)$$

Execute Accuracy (ExcAcc) Measures how often the translated FOL formulas lead to the correct reasoning outcome when executed by exact solver. This is analogous to logical reasoning accuracy but applied after translation:

$$\text{ExcAcc} = \frac{\# \text{ examples where translated FOL yields correct conclusion}}{\text{total } \# \text{ of examples}} \quad (5)$$

5.2 Dataset

We conduct our experiments on the FOLIO dataset [5], which is designed for evaluating FOL reasoning from natural language. FOLIO consists of 1,435 examples collected from two sources: WikiLogic (304 stories) and HybLogic (183 stories). Each instance in FOLIO consists of a set of premises and a conclusion expressed in natural language and FOL, along with a corresponding label indicating whether the conclusion is *True*, *False*, or *Uncertain* given the premises.

The dataset exhibits diverse logical complexity: reasoning depth ranges from 0 to 14, with the mode at 4, and 28.7% of examples requiring five or more reasoning steps to infer the conclusion. The NL sentences have an average of 9.86 words, and the total vocabulary consists of 4,351 words, demonstrating both linguistic richness and reasoning challenge. Table 1 summarizes these statistics.

Table 1: Statistics of the FOLIO dataset

Source	#Stories	#Premises	#Conclusions	NL Words Avg.
WikiLogic	304	1,353	753	8.50
HybLogic	183	1,054	682	11.52
Total	487	2,407	1,435	9.86

5.3 Experimental Results

5.3.1 Performance evaluation on Logical reasoning

We first evaluate the models on the natural language reasoning task with FOL. The experimental results are summarized in Table 2.

Table 2: Performance comparison on the FOLIO dataset

Algorithm	Model	Acc (%)	Running Time (s)
LLM reasoning	Qwen2.5:3b	36.27	3.13
LLM reasoning	Qwen2.5:14b	52.94	25.48
LoT [2]	Llama3.1	35.30	10.05
LoT [2]	Ministral-3:8b	42.60	44.11
LoT [2]	Qwen2.5:14b	58.30	26.47
SeRAG (Ours)	Qwen2.5:14b	63.73	57.22
ST-RAG (Ours)	Qwen2.5:14b	64.22	80.06

Both SeRAG and ST-RAG consistently outperform the LLM-only baselines as well as the LoT [2] method, demonstrating that retrieving logically similar examples effectively helps the model select more accurate truth values. Among all methods, ST-RAG achieves the highest accuracy, indicating that combining retrieval with step-by-step reasoning further improves logical inference performance.

5.3.2 Performance evaluation on NL-FOL translation

The second experiment evaluates model performance on the NL-FOL translation task using the FOLIO dataset. We compare naive LLM-based translation with our proposed step-by-step approaches, including ST-noRAG, ST-RAG without the fixer module, and the full ST-RAG architecture.

As shown in Table 3, the full ST-RAG model achieves the highest syntactic validity (93.13%), substantially outperforming both the LLM-based baselines and ST-noRAG. This result indicates that retrieving relevant translation examples via RAG significantly improves the generation of syntactically valid FOL expressions. Although ST-RAG without the fixer attains the highest execution accuracy (68.94%), its syntactic validity remains relatively low (78.92%). This suggests that high execution accuracy alone is insufficient, as syntactically invalid formulas may lead to non-generalizable reasoning behavior.

Table 3: Performance comparison for NL-FOL translation on the FOLIO dataset

Algorithm	Model	SynV (%)	ExcAcc (%)	Running Time (s)
LLM translation	Qwen2.5:3b	59.31	42.17	3.13
LLM translation	Qwen2.5:14b	71.08	62.07	25.48
ST-noRAG	Qwen2.5:14b	83.33	65.29	53.23
ST-RAG w/o Fixer	Qwen2.5:14b	78.92	68.94	56.66
ST-RAG (Ours)	Qwen2.5:14b	93.13	66.32	80.06

Overall, these results demonstrate that combining retrieval-based example selection with step-by-step translation and error correction substantially improves the robustness and reliability of NL-FOL translation.

5.4 Error Analysis

For SeRAG, we observe that the agent does not struggle with parsing the generated answers, as they only contain the final labels. Consequently, the predictions can be parsed reliably without additional post-processing.

In contrast, ST-RAG occasionally produces FOL syntax errors during the NL-FOL translation stage. A common category of errors that can be fixed by ST-RAG involves the use of unsupported operators or invalid predicate constructions, for example:

$$\forall animal (IsAnimal(animal) \rightarrow (\exists human Human(human) \wedge human = animal))$$

can be fixed as

$$\forall x (IsAnimal(x) \rightarrow Human(x))$$

or

$$JapaneseCompany(Game(TheLegendOfZelda)).$$

can be fixed as

$$Game(TheLegendOfZelda) \wedge \exists c (JapaneseCompany(c) \wedge ProducedBy(TheLegendOfZelda, c))$$

Such errors are typically local and limited in scope, and can be effectively corrected by the Fixer module. This is because the overall logical structure remains mostly intact, allowing the Fixer to repair minor syntactic or semantic violations.

However, there are cases where the current architecture fails to recover from translation errors. We identify two main failure patterns.

(1) Predicate arity mismatch. In some cases, the number of arguments of a predicate is inconsistent across different premises. For example:

$$HostedOlympicsAndParalympics(beijing, summer2008olympics, summer2008paralympics)$$

corresponding to: “Beijing hosted the 2008 Summer Olympics and 2008 Summer Paralympics Games.”

Meanwhile, another premise is translated as:

$$\begin{aligned} &HostedOlympicsAndParalympics(beijing, summer2008olympics) \\ &\wedge HostedWinterOlympics(beijing) \end{aligned} \tag{6}$$

for: “Beijing has hosted both the 2008 Summer Olympics and a winter olympics.”

In this scenario, the Fixer module is unable to resolve the inconsistency, as the error spans multiple predicates rather than being confined to a single expression. We hypothesize that this issue is partly caused by overly long predicate names, which increase the difficulty for the agent to maintain consistency across different premises.

(2) Overly complex and noisy logical forms. Another failure case occurs when the agent generates an excessively complex FOL formula containing multiple types of errors, including unsupported operators, redundant parentheses, and structural inconsistency. For example:

$$\begin{aligned} \exists s_1 \exists s_2 \exists s_3 \exists s_4 (&Season(s_1) \wedge Season(s_2) \wedge Season(s_3) \wedge Season(s_4) \\ &\wedge s_1 \neq s_2 \wedge s_1 \neq s_3 \wedge s_1 \neq s_4 \\ &\wedge s_2 \neq s_3 \wedge s_2 \neq s_4 \wedge s_3 \neq s_4 \\ &\wedge \forall x (Season(x) \rightarrow (SameAs(x, s_1) \vee SameAs(x, s_2) \vee SameAs(x, s_3) \\ &\vee SameAs(x, s_4)))) \end{aligned}$$

Such outputs are difficult to repair due to the accumulation of errors at different levels of the logical form.

Overall, while the proposed architecture is effective for premises with a small number of entities and can successfully fix local translation errors, it still struggles with sentences involving many subjects or highly complex logical structures. Handling global inconsistencies and large-scale structural errors remains an open challenge.

6 Discussion

This section discusses how the experimental results compare with our initial expectations, highlights the limitations of the proposed approaches, and outlines the lessons learned and future perspectives.

Overall, the experimental results show that the proposed methods consistently improve evaluation metrics compared to baseline approaches under the same experimental settings. The inclusion of the RAG module with step-by-step translation examples and the Fixer module was expected to enhance performance, and this expectation is largely confirmed by the results. Although the proposed pipelines are more time-consuming, the observed performance gains indicate that this computational trade-off is justified by the significant improvement in translation and reasoning quality.

Despite these encouraging results, several limitations remain. First, the retrieval examples used in the knowledge base are currently manually selected and tuned, which raises concerns about scalability and generalization to larger or more diverse datasets. Second, for the SeRAG framework, it remains unclear to what extent the reasoning process may be biased by the labels of the retrieved examples provided by the RAG module, potentially influencing the model’s decision-making behavior.

Another important observation concerns the evaluation metrics themselves. The current metrics may not fully capture the true quality of NL-to-FOL translation, as they primarily focus on syntactic validity and the correctness of the final logical conclusion. As a result, certain semantic errors or partial mismatches in meaning may not be adequately reflected in the reported scores.

7 Conclusion

This work presents two frameworks: (i) SeRAG, a framework for logical reasoning over FOL, and (ii) ST-RAG, which addresses the natural language-to-FOL translation problem by combining retrieval-augmented generation with a step-by-step translation knowledge base and a syntax correction module. Both methods substantially improve the evaluation metrics on the FOLIO dataset. Future work will focus on:

- **Model design improvement:** Expanding and automatically constructing the knowledge bases, and improving error correction.
- **Model fine-tuning:** Applying fine-tuning techniques such as LoRA for logical reasoning tasks.
- **Evaluation metrics:** Exploring or designing more reliable metrics for NL-FOL translation problems.

References

- [1] Ruikang Hu et al. “LTRAG: Enhancing autoformalization and self-refinement for logical reasoning with thought-guided RAG”. In: *Findings of the Association for Computational Linguistics: ACL 2025*. 2025, pp. 2483–2493.
- [2] Tongxuan Liu et al. “Logic-of-thought: Injecting logic into contexts for full reasoning in large language models”. In: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 2025, pp. 10168–10185.
- [3] Theo Olausson et al. “LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023, pp. 5153–5176.
- [4] Liangming Pan et al. “Logic-lm: Empowering large language models with symbolic solvers for faithful logical reasoning”. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. 2023, pp. 3806–3824.
- [5] Simeng Han et al. “Folio: Natural language reasoning with first-order logic”. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2024, pp. 22017–22031.
- [6] Ha-Thanh Nguyen et al. “A multi-step approach in translating natural language into logical formula”. In: *Legal Knowledge and Information Systems*. IOS Press, 2022, pp. 103–112.
- [7] Yu’an Yang et al. “Harnessing the power of large language models for natural language to first-order logic translation”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2024, pp. 6942–6959.
- [8] Yongchao Chen et al. “Nl2tl: Transforming natural languages to temporal logics using large language models”. In: *arXiv preprint arXiv:2305.07766* (2023).
- [9] Dongwei Jiang, Marcio Fonseca, and Shay B Cohen. “Leanreasoner: Boosting complex logical reasoning with lean”. In: *arXiv preprint arXiv:2403.13312* (2024).

- [10] Jundong Xu et al. “Faithful Logical Reasoning via Symbolic Chain-of-Thought”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 13326–13365. DOI: 10.18653/v1/2024.acl-long.720. URL: <https://aclanthology.org/2024.acl-long.720/>.
- [11] Patrick Lewis et al. “Retrieval-augmented generation for knowledge-intensive nlp tasks”. In: *Advances in neural information processing systems* 33 (2020), pp. 9459–9474.
- [12] Wenqi Fan et al. “A survey on rag meeting llms: Towards retrieval-augmented large language models”. In: *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*. 2024, pp. 6491–6501.
- [13] Jiachang Liu et al. “What makes good in-context examples for GPT-3?” In: *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd workshop on knowledge extraction and integration for deep learning architectures*. 2022, pp. 100–114.
- [14] Matthijs Douze et al. “The Faiss library”. In: (2024). arXiv: 2401.08281 [cs.LG].
- [15] Nikolaj Bjørner et al. “Programming Z3”. In: *International Summer School on Engineering Trustworthy Software Systems*. Springer, 2018, pp. 148–201.